

Techniques algorithmiques : dichotomie

Ambre Le Berre

MPSI

2025/2026

Plan du cours

1. Recherche d'un élément dans une liste
2. D'autres applications de la dichotomie

Recherche d'un élément dans une liste

Recherche “naïve”

```
def recherche(L, x):  
    """renvoie True si x est dans L, False sinon."""  
    for y in L:  
        if x == y:  
            return True  
    return False
```

Recherche dans une liste triée : la dichotomie

On a vu comment on peut trier une liste. Comment utiliser ce tri pour chercher un élément plus vite ?

Recherche dans une liste triée : la dichotomie

On a vu comment on peut trier une liste. Comment utiliser ce tri pour chercher un élément plus vite ?

Idée : on regarde au milieu de la liste.

- Si milieu $< x$, alors x est dans la deuxième moitié
- Sinon, dans la première moitié

Recherche dans une liste triée : la dichotomie

Recherche de “2” dans la liste [-1, 0, 1, 2, 3, 4, 5, 10, 123, 124]

-1	0	1	2	3	4	5	10	123	124
-1	0	1	2	3	4	5	10	123	124
-1	0	1	2	3	4	5	10	123	124
-1	0	1	2	3	4	5	10	123	124

Recherche dans une liste triée : la dichotomie

```
def recherche_dicho(L, x):  
    """Si L est une liste triée dans l'ordre croissant, renvoie  
    l'indice de x dans L, ou bien None si x n'est pas dans L."""  
    i = 0 #borne inf inclue  
    j = len(L) #borne sup exclue  
  
    while (j - i) > 1:  
        milieu = (i + j) // 2  
        if L[milieu] > x:  
            i = milieu + 1  
        elif L[milieu] < x:  
            j = milieu  
        else: #cas ou L[milieu] == x  
            return milieu  
    return None
```


Combien de tour de boucle au maximum pour une liste de taille 8 ? De taille 16 ? Et pour 2^n en général ?

Complexité

Combien de tour de boucle au maximum pour une liste de taille 8 ? De taille 16 ? Et pour 2^n en général ?

En fait, la complexité est **logarithmique**, soit en $O(\log(n))$.

D'autres applications de la dichotomie

Recherche du 0 d'une fonction

Supposons qu'on ait une fonction f quelconque et qu'on cherche où elle s'annule sur un intervalle $[a, b]$, tel que $f(a) < 0$ et $f(b) > 0$.

On peut appliquer le même principe !

Recherche du 0 d'une fonction

```
def recherche_zero(f, a, b, eps):  
    """Si  $f(a) < 0$  et  $f(b) > 0$ , renvoie un zero de  $f$  sur  $[a, b]$  à eps  
    près."""  
    while b - a > eps:  
        milieu = a + b / 2  
        if f(milieu) < 0:  
            a = milieu  
        else:  
            b = milieu  
    return (a + b) / 2
```

Version récursive d'un algorithme dichotomique

```
def recherche_zero(f, a, b, eps):  
    milieu = (a + b) / 2  
    if b-a <= eps:  
        return milieu  
    elif f(milieu) < 0:  
        return recherche_zero(f, milieu, b, eps)  
    else:  
        return recherche_zero(f, a, milieu, eps)
```

Exponentiation rapide

```
def exp_rapide(x, n):  
    """Calcule  $x^n$  en  $O(\log(n))$ """  
    c = 1  
    while n != 0:  
        if n % 2 == 0:  
            n /= 2  
            x *= x  
        else:  
            c *= x  
            x -= 1  
    return c
```

Tri fusion (ou “partition-fusion”)

```
def fusion(L1, L2):  
    """fusionne deux listes triées en ordre croissant en une seule  
    liste triée également."""  
    resultat = []  
    i1 = 0  
    i2 = 0  
    while i1 < len(L1) or i2 < len(L2):  
        if i1 < len(L1) and \  
            (i2 == len(L2) or L1[i1] < L2[i2]):  
            resultat.append(L1[i1])  
            i1 += 1  
        else:  
            resultat.append(L2[i2])  
            i2 += 1  
    return resultat
```


Tri fusion (ou “partition-fusion”)

```
def tri_fusion(L):  
    if len(L) <= 1:  
        return L[:]  
    milieu = len(L)//2  
  
    #partition  
    L1 = L[:milieu]  
    L2 = L[milieu:]  
  
    #tri récursif  
    L1 = tri_fusion(L1)  
    L2 = tri_fusion(L2)  
  
    #fusion  
    return fusion(L1, L2)
```

Tri rapide

```
def tri_rapide(L):  
    if len(L) <= 1:  
        return L[:]  
    pivot = L[0]  
    plus_petit = []  
    plus_grand = []  
    for x in L[1:]:  
        if x < pivot:  
            plus_petit.append(x)  
        else:  
            plus_grand.append(x)  
    plus_petit = tri_rapide(plus_petit)  
    plus_grand = tri_rapide(plus_grand)  
    return plus_petit + [pivot] + plus_grand
```